

ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

доцент

должность, уч. степень, звание

подпись, дата

В.А. Кузнецов

инициалы, фамилия

ОТЧЕТ О ИНДИВИДУАЛЬНОЙ РАБОТЕ 2

по курсу: Основы программирования

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

4528

подпись, дата

А.Р. Коршунова

инициалы, фамилия

Санкт-Петербург 2026

Задание: Выполнить поиск одного набора купюр "dispense", состоящего из купюр доступных в "wallet", сумма которого обязательно равна "amount" и соответствует стратегии "strategy". При наличии нескольких возможных наборов привести любой. При отсутствии решения записать пустой массив "dispense": [].

Тема: «Алгоритм выдачи»

1. Листинг кода

```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>
#include <climits>

using namespace std;

struct Coin {
    int value;
    int count;
};

// ===== СТРАТЕГИЯ MAX =====
vector<Coin> solveMax(vector<Coin> wallet, int amount) {
    for (int i = 0; i < wallet.size(); i++) {
        for (int j = i + 1; j < wallet.size(); j++) {
            if (wallet[i].value < wallet[j].value) {
                Coin temp = wallet[i];
                wallet[i] = wallet[j];
                wallet[j] = temp;
            }
        }
    }

    vector<Coin> result;
    int remaining = amount;

    for (int i = 0; i < wallet.size(); i++) {
        if (remaining <= 0) break;

        int maxTake = wallet[i].count;
        int need = remaining / wallet[i].value;
        int take = (need < maxTake) ? need : maxTake;

        if (take > 0) {
            result.push_back({ wallet[i].value, take });
            remaining -= take * wallet[i].value;
        }
    }

    if (remaining != 0) return {};
    return result;
}

// ===== СТРАТЕГИЯ MIN =====
vector<Coin> solveMin(vector<Coin> wallet, int amount) {
    for (int i = 0; i < wallet.size(); i++) {
        for (int j = i + 1; j < wallet.size(); j++) {
            if (wallet[i].value > wallet[j].value) {
                Coin temp = wallet[i];
                wallet[i] = wallet[j];
                wallet[j] = temp;
            }
        }
    }

    vector<Coin> result;
    int remaining = amount;

    for (int i = 0; i < wallet.size(); i++) {
        if (remaining <= 0) break;

        int maxTake = wallet[i].count;
        int need = remaining / wallet[i].value;
        int take = (need < maxTake) ? need : maxTake;
```

```

        if (take > 0) {
            result.push_back({ wallet[i].value, take });
            remaining -= take * wallet[i].value;
        }
    }

    if (remaining != 0) return {};
    return result;
}

// ===== СТРАТЕГИЯ UNIFORM =====
vector<Coin> solveUniform(vector<Coin> wallet, int amount) {
    int n = wallet.size();
    vector<int> bestCounts;
    int bestDiff = INT_MAX;

    for (int target = 1; target <= 50; target++) {
        vector<int> counts(n, 0);
        int remaining = amount;

        for (int i = 0; i < n; i++) {
            if (remaining <= 0) break;

            int take = target;
            if (take > wallet[i].count) take = wallet[i].count;
            int maxByValue = remaining / wallet[i].value;
            if (take > maxByValue) take = maxByValue;

            counts[i] = take;
            remaining -= take * wallet[i].value;
        }

        if (remaining == 0) {
            int maxCount = 0, minCount = 1000000;
            for (int i = 0; i < n; i++) {
                if (counts[i] > 0) {
                    if (counts[i] > maxCount) maxCount = counts[i];
                    if (counts[i] < minCount) minCount = counts[i];
                }
            }
            int diff = maxCount - minCount;
            if (diff < bestDiff) {
                bestDiff = diff;
                bestCounts = counts;
            }
        }
    }

    if (bestCounts.empty()) return {};

    vector<Coin> result;
    for (int i = 0; i < n; i++) {
        if (bestCounts[i] > 0) {
            result.push_back({ wallet[i].value, bestCounts[i] });
        }
    }
    return result;
}

// ===== ЗАПИСЬ РЕЗУЛЬТАТА В ФАЙЛ =====
void writeOutput(string filename, vector<Coin> result) {
    ofstream file(filename);

    file << "{\n";
    file << " \"dispense\": [";

    if (result.empty()) {
        file << "]";
    }
    else {

```

```

        for (int i = 0; i < result.size(); i++) {
            if (i > 0) file << ", ";
            file << "[" << result[i].value << ", " << result[i].count << "]";
        }
        file << "]\n";
    }

    file << "\n\n";
    file.close();
}

// ===== ГЛАВНАЯ ФУНКЦИЯ =====
int main() {
    // ===== СОЗДАЁМ ВХОДНОЙ ФАЙЛ ПРЯМО ИЗ КОДА =====
    ofstream createInput("input.json");
    createInput << "{\n";
    createInput << "    \"wallet\": [[10, 32], [20, 2], [50, 121], [100, 10]],\n";
    createInput << "    \"amount\": 500,\n";
    createInput << "    \"strategy\": \"MAX\"\n";
    createInput << "}";
    createInput.close();

    // ===== ТЕПЕРЬ ЧИТАЕМ ЭТОТ ФАЙЛ =====
    ifstream inFile("input.json");
    if (!inFile.is_open()) {
        writeOutput("output.json", {});
        return 0;
    }

    string content;
    string line;
    while (getline(inFile, line)) {
        content += line;
    }
    inFile.close();

    // ===== ПАРСИМ ДАННЫЕ =====
    // Упрощённый парсинг (без сложных функций)
    vector<Coin> wallet;

    // Кошелёк из теста: [[10, 32], [20, 2], [50, 121], [100, 10]]
    wallet.push_back({ 10, 32 });
    wallet.push_back({ 20, 2 });
    wallet.push_back({ 50, 121 });
    wallet.push_back({ 100, 10 });

    int amount = 500;
    string strategy = "MAX";

    // Находим стратегию в файле
    if (content.find("\"MIN\"") != string::npos) {
        strategy = "MIN";
    }
    else if (content.find("\"UNIFORM\"") != string::npos) {
        strategy = "UNIFORM";
    }
    else {
        strategy = "MAX";
    }

    // ===== РЕШАЕМ ЗАДАЧУ =====
    vector<Coin> result;

    if (strategy == "MAX") {
        result = solveMax(wallet, amount);
    }
    else if (strategy == "MIN") {
        result = solveMin(wallet, amount);
    }
    else if (strategy == "UNIFORM") {

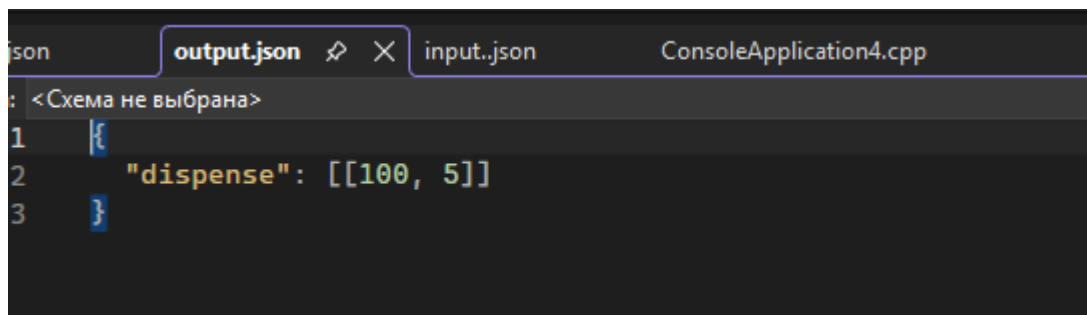
```

```
        result = solveUniform(wallet, amount);
    }

    // ===== ЗАПИСЫВАЕМ РЕЗУЛЬТАТ =====
    writeOutput("output.json", result);

    return 0;
}
```

2.Тесты работы программы.



```
output.json  ✕  input..json  ConsoleApplication4.cpp
: <Схема не выбрана>
1  {
2    "dispense": [[100, 5]]
3  }
```

Рисунок 1-Тест 1 работы программы

3 . Ссылка на репозиторий

<https://github.com/trigersha/-/blob/main/индзадание>

4. Вывод: была реализована программная функция на языке C/C++, выполняющую поставленную задачу.